

3_serwisy_ćwiczenia

March 22, 2022

```
[17]: import rospy
      rospy.init_node("serwis_test")
```

1 Klient

Dla podanych nazw serwisów należy utworzyć klientów, którzy wyślą i odbiorą zapytanie z serwera. Do serwisów testowych wykorzystana została symulacja z turtlesim i 3 turtlebotami z wprowadzenia.

1.1 /kill

```
[3]: !rosservice type /kill
```

```
turtlesim/Kill
```

```
[5]: !rosservice info /kill
```

```
Node: /turtlesim
URI: rosrpc://localhost:55583
Type: turtlesim/Kill
Args: name
```

```
[6]: !rossrv show turtlesim/Kill
```

```
string name
---
```

```
[18]: from turtlesim.srv import Kill, KillRequest, KillResponse
```

```
[19]: kill = rospy.ServiceProxy("kill", Kill)
```

```
[21]: request = KillRequest()
      request.name = "t8"
```

```
[22]: response = kill(request)
```

```
[23]: response
```

```
[23]:
```

1.2 /spawn

```
[7]: !rosservice type /spawn
```

```
turtlesim/Spawn
```

```
[8]: !rossrv show turtlesim/Spawn
```

```
float32 x  
float32 y  
float32 theta  
string name  
---  
string name
```

```
[25]: from turtlesim.srv import Spawn, SpawnRequest, SpawnResponse  
      # alternatywnie  
      # from turtlesim.srv import *
```

```
[26]: spawn = rospy.ServiceProxy("spawn", Spawn)
```

```
[27]: request = SpawnRequest()  
      request.name = "t9"  
      request.x = 2  
      request.y = 3
```

```
[28]: response = spawn(request)
```

```
[29]: print(response)
```

```
name: "t9"
```

1.3 /turtle1/set_pen

```
[9]: !rosservice type /turtle1/set_pen
```

```
turtlesim/SetPen
```

```
[10]: !rossrv show turtlesim/SetPen
```

```
uint8 r
uint8 g
uint8 b
uint8 width
uint8 off
---
```

```
[33]: from turtlesim.srv import SetPen, SetPenRequest, SetPenResponse
```

```
[34]: set_pen = rospy.ServiceProxy("/turtle1/set_pen", SetPen)
```

```
[36]: request = SetPenRequest()
      request.r = 255
      request.off = 0
      request.width = 7

      set_pen(request)
```

```
[36]:
```

```
[ ]: # z zapisaniem odpowiedzi
      response = set_pen(request)
```

1.4 /turtle1/teleport_absolute

```
[13]: !rosservice type /turtle1/teleport_absolute
```

```
turtlesim/TeleportAbsolute
```

```
[14]: !rossrv show turtlesim/TeleportAbsolute
```

```
float32 x
float32 y
float32 theta
---
```

```
[37]: from turtlesim.srv import TeleportAbsolute, TeleportAbsoluteRequest
```

```
[38]: teleport = rospy.ServiceProxy("/turtle1/teleport_absolute", TeleportAbsolute)
```

```
[39]: request = TeleportAbsoluteRequest()
      request.x = 4
      request.y = 8
      request.theta = 2
```

```
teleport(request)
```

[39]:

[]:

1.5 /tb3_1/move_base/make_plan

[15]: `rosservice type /tb3_1/move_base/make_plan`

```
nav_msgs/GetPlan
```

[16]: `rossrv show nav_msgs/GetPlan`

```
geometry_msgs/PoseStamped start
  std_msgs/Header header
    uint32 seq
    time stamp
    string frame_id
  geometry_msgs/Pose pose
    geometry_msgs/Point position
      float64 x
      float64 y
      float64 z
    geometry_msgs/Quaternion orientation
      float64 x
      float64 y
      float64 z
      float64 w
geometry_msgs/PoseStamped goal
  std_msgs/Header header
    uint32 seq
    time stamp
    string frame_id
  geometry_msgs/Pose pose
    geometry_msgs/Point position
      float64 x
      float64 y
      float64 z
    geometry_msgs/Quaternion orientation
      float64 x
      float64 y
      float64 z
      float64 w
float32 tolerance
```

```

---
nav_msgs/Path plan
  std_msgs/Header header
    uint32 seq
    time stamp
    string frame_id
  geometry_msgs/PoseStamped[] poses
    std_msgs/Header header
      uint32 seq
      time stamp
      string frame_id
    geometry_msgs/Pose pose
      geometry_msgs/Point position
        float64 x
        float64 y
        float64 z
      geometry_msgs/Quaternion orientation
        float64 x
        float64 y
        float64 z
        float64 w

```

2 Serwer

Dla podanych typów wiadomości serwisowych utworzyć odpowiednie serwery

2.1 pkg_tsr/Task

-> współrzędne punktu, rysowanie kwadratu

2.2 std_srvs/Empty

-> czyszczenie_mapy (to samo co clear, ale napisany własny serwer)

```
[40]: !rossrv show std_srvs/Empty
```

```
---
```

```
[41]: from std_srvs.srv import Empty, EmptyResponse

def czyszczenie_mapy(req):
    clear_map = rospy.ServiceProxy('clear', Empty)
    clear_map()
```

```

    return EmptyResponse()

server = rospy.Service("czyszczenie_mapy", Empty, czyszczenie_mapy)

```

2.3 std_srvs/SetBool

```
[59]: !rossrv show std_srvs/SetBool
```

```

bool data
---
bool success
string message

```

```
[58]: from std_srvs.srv import SetBool, SetBoolResponse
      from std_srvs.srv import Empty
```

```
[60]: def reset(req):
      response = SetBoolResponse()
      if req.data:
          # wykona się restart symulacji
          reset_sim_state = rospy.ServiceProxy('reset', Empty)
          reset_sim_state()
          response.success = 1
          response.message = "Restart pomyslny"
      else:
          # nie wykona się nic, czyszczenie zakończone niepowodzeniem
          response.success = 0
          response.message = "Bład restartu"
      return response
```

```
[61]: server_restart = rospy.Service("reset_mapy", SetBool, reset)
```

```
[ ]:
```

2.4 std_srvs/Trigger

```
[43]: !rosservice type /turtle1/teleport_absolute
```

```
turtlesim/TeleportAbsolute
```

```
[46]: !rossrv show Trigger
```

```

[std_srvs/Trigger]:
---

```

```
bool success
string message
```

```
[44]: from std_srvs.srv import Trigger, TriggerResponse
      from turtlesim.srv import TeleportAbsolute
```

```
[56]: def rys_troj(req):
      set_pose = rospy.ServiceProxy('/turtle1/teleport_absolute', TeleportAbsolute)
      clear_map = rospy.ServiceProxy('clear', Empty)
      clear_map()

      request = TeleportAbsoluteRequest()
      request.x = 2
      request.y = 1
      request.theta = 2
      set_pose(request)

      request.x = 5
      request.y = 7
      request.theta = 2
      set_pose(request)

      request.x = 2
      request.y = 7
      request.theta = 2
      set_pose(request)

      response = TriggerResponse()
      response.success = 1
      response.message = "DONE"
      return response
```

```
[57]: server = rospy.Service("/turtle1/trigger_triangle", Trigger, trigger)
```

```
[55]: server.shutdown()
```

```
[ ]: # wersja 2
      from std_srvs.srv import Trigger, TriggerResponse, Empty
      from turtlesim.srv import TeleportAbsolute, TeleportAbsoluteRequest

      def rys_troj(request):
          teleport = rospy.ServiceProxy("/turtle1/teleport_absolute", TeleportAbsolute)
          clear_map = rospy.ServiceProxy('clear', Empty)

          request = TeleportAbsoluteRequest()
          request.x = 2
```

```

request.y = 1
request.theta = 37
teleport(request)
clear_map()

request.x = 4
request.y = 2
request.theta = 69
teleport(request)

request.x = 4
request.y = 3
request.theta = 4.2
teleport(request)

request.x = 2
request.y = 1
request.theta = 37
teleport(request)

response = TriggerResponse()
response.success = 1
response.message = "Trzeba robić package"
return response

serwer = rospy.Service("/rysowanie_trojkata", Trigger, rys_troj)

```